LC 238 — Product of Array Except Self.

Input 1 2 0 8 4 0

64

x _____

_ x _____

_____ x _____

_____ x _

_____ x

64 32

Assume without Zero

Input 1 2 -5 2 4

-80

x _____

_ x _____

_____ x _____

_____ x _

_____ x

-80 -40 16 -40 -20

2 issues when there are zeroes

① Array product will be zero → hence divide logic cant work.

② When traversing → we cant put zeroes directly.

	1			4				8
1	0	3	5	0	2	3	4	0

First Pass O(n).

① total_prod = 360 ignoring zeroes.

② is_zero = true.

③ zero posi = [1, 4, 8].

[3.2] posi_pointer ++ ← at every insertion.

Filling ANSWER

① Lookup_pointer = 0

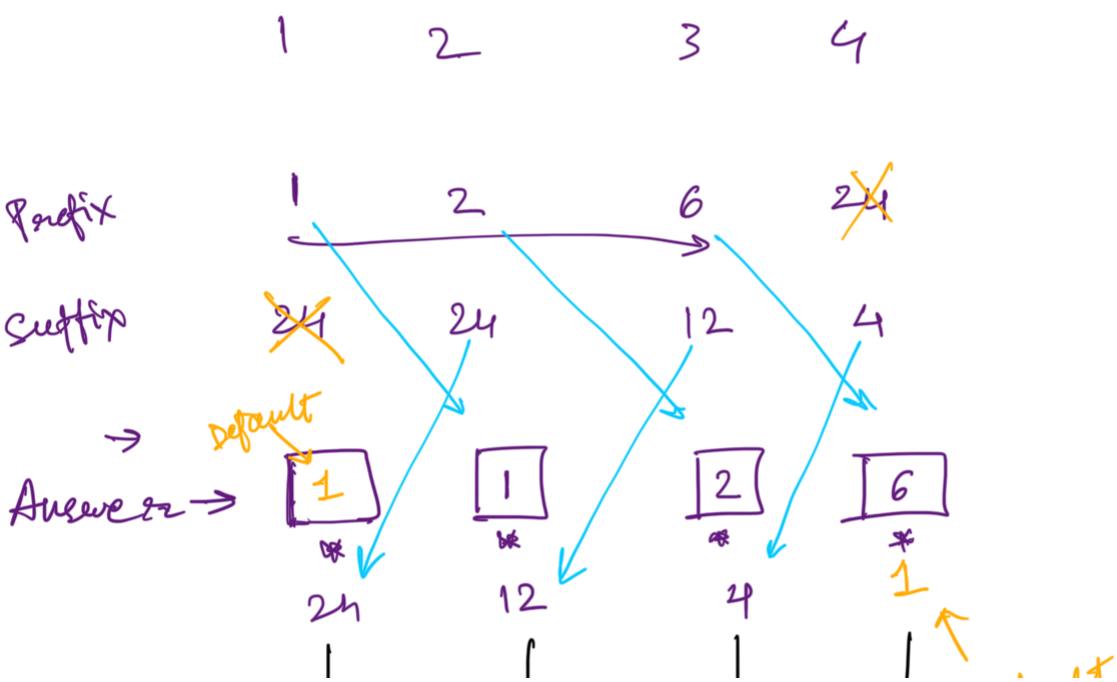
②

if zero_count > 1 → return arr(n, 0)

size n | All zeroes

if zero-count = return array with all zeroes & total-prod at zero_posi.

if zero-count = 0 → simple divide logic.



Default

Final Answer	24	12	8	6

LC - 56

Merge Intervals

[[1, 2] [8, 10] [1, 9]]

↓

[1, 9] [8, 10]

↓

[1, 10]

1, 1, 2, 8, 9, 10

1, 2, 3, 6, 8, 10, 15, 18

START

[[1, 3], [2, 6], [8, 10]]

Final Pairs.

[1, 6]

Opening =1

Check closing(i), opening(i+1)

< → merge

l++, r++

Compare & choose bigger

∴ closing = 6

$i=0$. $i=1$ $i=2$.

[[1, 3], [2, 6], [8, 10]]

Final pairs.

[1, 6]

$i=0$

$s = 1$

$e = 3$

$s = 1$

$e = \max(3, 6)$

$= 6$.

$i=1$.

$s = 2$

$e = 6$ } can be accumulate?

yes

$\therefore$ continue

$i=2$.

$s = 8$

$e = 10$.

comp FAILS → bcoz no $(i+1)$

pushback.

LC 75 - Sort Colours. (0, 1, 2).

Brute → Sorting Algo. like Quicksort. or Merge Sort.

Better → Count freq^n & edit the array.

Optimal → DUTCH NATIONAL FLAG algorithm.

We will be having 3 pointers low, mid & high.

Rules:

[0, ..., low − 1] → 0s

[low, ..., mid − 1] → 1s

[high + 1, ... n − 1] → 2s

Golden Rule

[mid, ... high] → Unsorted elements

Algo

- we run the algo on "mid"

1. Initialize low = 0 , mid = 0 , high = n - 1 .

2. while mid <= high ← since failing this will mean that there is NO unsorted part.

   2.1 if a[mid] == 0

   → swap (arr[low] , arr[mid])
   → low ++
   → mid ++

   2.2 if arr[mid] == 1

   → mid ++

   2.3 if arr[mid] == 2

   → swap (arr[mid] , arr[high])
   → high - -